NORTH BENGAL INTERNATIONAL UNIVERSITY

Lab Report : 1

TOPIC : PYTHON FUNDAMENTALS, IF-ELSE STATEMENTS, FOR, WHILE LOOPS, LIST AND PYTHON TUPLES IN PYTHON.

COURSE CODE : MATH 0541-2212

COURSE TITLE : NUMERICAL ANALYSIS SESSIONAL

Submitted By:

MD. TANVIR RAHMAN PRANTO

0712340012101005

CSE -26

Submitted To:

SAIFUR RAHMAN

LECTURER, CSE

Theory: Python is a versatile, high-level programming language known for its simplicity and readability. Created by Guido van Rossum and first released in 1991, it emphasizes clear and concise syntax, making it an excellent choice for beginners and experienced developers alike. Python supports multiple programming paradigms.

Key Feature of python:

Certainly! Here are some key features of Python:

- **Easy to Read and Write:** Python's syntax is clear and concise, making it accessible to beginners and allowing experienced developers to write code more efficiently.
- **Interpreted Language:** Python code is executed line-by-line, which makes debugging easier and speeds up development cycles.
- **Dynamically Typed:** Variables in Python do not need explicit declarations, allowing for more flexibility in coding.
- **Extensive Standard Library:** Python comes with a rich standard library that provides tools and modules for various tasks, from web development to data manipulation.
- **Cross-Platform Compatibility:** Python runs on many platforms, including Windows, macOS, and Linux, allowing developers to write code that works across different environments.
- **Support for Multiple Programming Paradigms:** Python supports procedural, object-oriented, and functional programming, making it versatile for different types of projects.
- **Large Community and Ecosystem:** Python has a vast and active community, providing a wealth of libraries, frameworks, and tools to enhance development.
- **Robust Frameworks and Libraries**: Popular libraries such as NumPy, Pandas, and TensorFlow extend Python's capabilities for scientific computing, data analysis, and machine learning.
- **Integration Capabilities:** Python can easily integrate with other languages like C, C++, and Java, as well as web services and databases.
- **Ideal for Automation:** Python is often used for scripting and automating repetitive tasks, improving productivity and efficiency.

Common Use case: Python is widely used in various domains due to its versatility and ease of use. It is a staple in web development, with frameworks like Django and Flask. In data science and analytics, libraries such as Pandas and NumPy enable data manipulation and analysis. Python's simplicity and extensive libraries make it a favorite for machine learning and artificial intelligence, with tools like TensorFlow and scikit-learn. It's also popular in automation, scripting, and testing, improving productivity by automating repetitive tasks. Python's cross-platform nature ensures it is a go-to language for software development and integration tasks across different systems.

Python Variables: In Python, a variable is like a name tag that you attach to a value to keep track of it. For example, if you want to store the number 10, you can use a variable like this: `x = 10`. Now, every time you use `x`, Python will know you're referring to the value 10. Variables can hold different types of values, such as numbers, words, or lists,

and you can change what value they hold at any time. They make it easy to manage and use information in your program.

If-Else Statement: An if-else statement in programming allows you to make decisions based on conditions. If the condition is true, one block of code runs. If the condition is false, a different block of code runs. This helps the program choose different actions depending on what is happening.

For loop: A for loop is a way to repeat a block of code multiple times. It starts with an initial value, checks a condition, and updates the value each time the loop runs. The code inside the loop keeps running until the condition is no longer true. This makes it easy to repeat actions like going through a list of items.

List: A list in Python is like a box where you can store multiple items, such as numbers or words, in order. You can create a list by putting these items inside square brackets, separated by commas. Lists are helpful because you can easily add, remove, or change the items inside the box, making it flexible and useful for managing different kinds of data.

String: In Python, a string is a sequence of characters enclosed in quotes, either single `'` or double `"`. Strings are used to represent text and can include letters, numbers, symbols, and spaces. They are immutable, meaning once a string is created, it cannot be changed. However, you can perform various operations on strings, such as concatenation, slicing, and formatting, to manipulate and use text effectively in your programs. For example, `name = "Alice"` assigns the string "Alice" to the variable `name`.

Touple: A tuple in Python is an ordered collection of elements that is immutable, meaning the items inside cannot be changed after the tuple is created. Tuples are defined using parentheses `()`, with items separated by commas. They are used to group related data together and can hold multiple data types. For example, `(1, "apple", 3.14)` is a tuple containing an integer, a string, and a float. Tuples are often used when you want to ensure the data remains constant throughout the program.

Code

If-Else Statements Code:

```
result = input('Enter your GPA: ')

if float(result) > 3.0:
    print("Congratulations! You did great!")
else:
    print("Better luck next time.")
```

Output:

```
OUTPUT    TERMINAL

PS C:\Users\Lenovo> & C:/Users/Lenovo/AppData/Local/Programs/Python/Pytho
cuments/Python/lab1.py
Enter your GPA: 2.9
Better luck next time.
PS C:\Users\Lenovo>
```

For Loop code:

```
for i in range(1, 6):
    print(i)
```

Output:

```
OUTPUT    TERMINAL

PS C:\Users\Lenovo> & C:/Users/Lenovo/AppData/Local/Programs/Python/Python313/p
cuments/Python/lab1.py
1
2
3
4
5
PS C:\Users\Lenovo>
```

List Code:

```
fruits = ["apple", "banana", "cherry"]
print(fruits)
```

Output:

```
OUTPUT    TERMINAL

PS C:\Users\Lenovo> & C:/Users/Lenovo/AppData/Local/Programs/Python/
cuments/Python/lab1.py
['apple', 'banana', 'cherry']
PS C:\Users\Lenovo>
```

String Code:

```
message = "Hello, World!"
print(message)
```

output:

```
OUTPUT    TERMINAL

PS C:\Users\Lenovo> & C:/Users/Lenovo/AppData/Local/Programs/Python/Python313/
cuments/Python/lab1.py
Hello, World!
PS C:\Users\Lenovo>
```

Tuple code:

```
my_tuple = (1, "hello", 3.14)
print(my_tuple)
print(my_tuple[0])
```

```python
print(my_tuple[1])
print(my_tuple[2])
```

Output:

```
OUTPUT    TERMINAL

PS C:\Users\Lenovo> & C:/Users/Lenovo/AppData/Local/Programs/Python/Python313/
cuments/Python/lab1.py
(1, 'hello', 3.14)
1
hello
3.14
PS C:\Users\Lenovo>
```

Conclusion: In conclusion, understanding the basics of Python is essential for any aspiring programmer. If-else statements allow for decision-making in your code by executing different blocks of code based on certain conditions. For loops enable you to iterate over sequences like lists or ranges, making repetitive tasks more manageable. Lists in Python provide a flexible way to store and manipulate ordered collections of items, while tuples offer a way to create immutable sequences of data. Together, these fundamental concepts form the building blocks of Python programming, empowering you to write efficient, clear, and dynamic code.